Int. j. inf. tecnol. (September 2025) 17(7):3859–3875
https://doi.org/10.1007/s41870-025-02597-8

ORIGINAL RESEARCH

HawkPhish-DNN cybersecurity model: adaptive hybrid optimization and deep learning for enhanced multi-objective phishing URL detection

Saif Ali Abd Alradha Alsaidi[1] · Husam Jasim Mohammed[2] · Riyadh Rahef Nuiaa Al Ogaili[1] · Zeinab Ali Dashoor[3] · Ali Hakem Alsaeedi[4] · Dhiah Al-Shammary[4] · Ayman Ibaida[5]

Received: 12 February 2025 / Accepted: 4 June 2025 / Published online: 30 June 2025
© The Author(s) 2025

Abstract Phishing attacks continue to pose significant security risks, necessitating effective detection methods to safeguard users. This paper presents HawkPhish-DNN cybersecurity model, a novel phishing detection framework that integrates Harris Hawk Optimization (HHO) with a Deep Neural Network (DNN). In the preprocessing phase, redundant URLs and domain features are removed, while URL length and entropy are extracted to form an efficient feature set. The detection model employs advanced neural layers, including Sigmoid and ReLU, to enhance learning and classification. By leveraging multi-objective HHO, HawkPhish-DNN optimizes accuracy and mitigates false positives through a time-varying penalty function, Pareto dominance, and crowding distance strategies. Empirical findings demonstrate that HawkPhish-DNN cybersecurity model achieves an accuracy of up to 99.6% and a false positive rate as low as 0.2% on benchmark datasets. Additionally, it maintains low computational overhead, ensuring practicality for real-time deployment. These results highlight the potential of HawkPhish-DNN cybersecurity model in providing a robust, user-friendly defense against phishing threats without inflating false alarms.

Keywords HawkPhish-DNN cybersecurity model · Phishing · DNN · HHO · Multi-objective

1 Introduction

The internet-based services have deeply transformed the manner in which people and organizations conduct their business, which is much more convenient. But the increase in online activity has also increased the risks concerning cyber threats; among the most common is phishing [1, 2]. To conduct phishing attacks, cybercriminals impersonate a legitimate authority to extract login details, financial information, and personal identification from the users [3, 4] [5]. The sophistication of these attacks has increased significantly, enabling cybercriminals to craft fake websites that resemble reputable organizations. These acts are possible

✉ Ayman Ibaida
ayman.ibaida@vu.edu.au

Saif Ali Abd Alradha Alsaidi
salsaidi@uowasit.edu.iq

Husam Jasim Mohammed
husam@kus.edu.iq

Riyadh Rahef Nuiaa Al Ogaili
riyadh@uowasit.edu.iq

Zeinab Ali Dashoor
zdashoor@uowasit.edu.iq

Ali Hakem Alsaeedi
ali.alsaeedi@qu.edu.iq

Dhiah Al-Shammary
d.alshammary@qu.edu.iq

[1] College of Computer Science and Information Technology, Wasit University, Al-Kut 52001, Wasit, Iraq

[2] Department of Environmental Health Science, College of Energy and Environmental Science, Al-Karkh University of Science, Baghdad 10011, Iraq

[3] College of Administration and Economics, Wasit University, Al-Kut 52001, Wasit, Iraq

[4] College of Computer Science and Information Technology, University of Al-Qadisiyah, Dewaniyah, Iraq

[5] Intelligent Technology Innovation Lab, Victoria University, Melbourne 3011, Australia

due to the trust and reliance users place in online platforms [6, 7].

Phishing threats manipulate the basic human nature of trusting people and the weaknesses presented by the channels of digital communication. In these attacks, cybercriminals attempt to create trust through email tampering, domain name system (DNS) phishing, and impersonation using HTTPS certificates [8]. Phishing poses a serious problem for many defenses because, unlike a rule-based heuristic or blacklist based system, these methods of system protection cannot compete with the speed at which these attacks adjust and change their strategy [9, 10]. Phishing scams have dramatically advanced in their complexity in recent years. Focusing on attempts based on social engineering, these systems need complete rethinking of the approaches taken for protection against such malwares. Protecting today's systems requires more sophisticated, flexible, and intelligent defenders that detect the cybercrime in real-time and fend it off immediately [11–13].

To overcome these constraints, more recent machine learning (ML) and deep learning (DL) approaches provide better pattern recognition and anomalies detection features [14–17]. Phishing threats detection is further enhanced by ML and DL models which process terabytes of information and identify even the subtle and complex phishing patterns more accurately with less false positives[18] [19]. The URLs of certain websites can be analyzed using lexical host and webpage content features to tell apart benign and malicious sites. Nevertheless, some researchers have pointed out drawbacks to these methods[20]. A high level of false positive poses a threat to user confidence in security systems as well as clear annoyance and, the ever-evolving phishing techniques like URL shortening services, domain generation algorithms, and internationalized domain names can greatly minimize the effectiveness of ML and DL based detectors[21, 22] [23].

Aspects that influence the performance of the phishing detector's DL are the parameterization of the network in terms of the weights and biases it possess and its optimization [24–26]. Obstructions to achieving convergent functions in deep neural networks often appear from traditional optimization techniques, including stochastic gradient descent (SGD). Those obstructions would include slow convergence rates, high susceptibility to overfitting, and getting caught in a web of local minima: all of which abandon the model's ability to effectively generalize to a new set of data. In order to resolve the problems mentioned, alternative methods that efficiently explore the parameter space, evade local minima, and simultaneously improve the performance of the DNN dynamics is required [27, 28].

In addressing these requirements, this study proposes the 'HawkPhish-DNN' cybersecurity model which incorporates the HHO into a DNN model to improve phishing URL detection. The Harris hawk's cooperative hunting techniques inspire the HHO algorithm, which offers enhanced optimization capabilities to competently balance exploration and exploitation during the search process. Throughout the parameter space of the DNN, the HHO algorithm uses the sophisticated and dynamic behaviors of hawks while hunting to control and optimize the network's parameters associated with the weights and biases of the neurons to classify at the highest levels. The integration of these structural components eliminates the traditional optimization approaches' weaknesses while simultaneously elevating critical performance metrics such as accuracy, precision, recall, F1-score, and Area Under the Curve (AUC).

This work presents notable contributions to both the areas of cybersecurity as well as those of machine learning:

- The Novel Combination of HHO and DNN: This work shows for the first time the fusion of HHO with DNN. Phishing detection using this combination was more effective than the traditional DNN usage. This approach effectively improves both optimization and classification accuracy by leveraging the features inherent within both HHO and DNN.
- HawkPhish-DNN "Phishing Detection System" Advanced Feature Extraction and Preprocessing: The model also comes with advanced features extracted through methods such as entropy extraction, TF-IDF vectorization and lexical analysis. These techniques improve the model as they allow it to integrate a higher level representation of the input data, thus capturing important lexical and semantic features for reliable determination of whether a URL is a phishing website or a genuine one regardless of the obfuscation techniques employed by the phishing attacks.
- Using SMOTE to Solve Class Imbalance Problem: The incorporation of the Synthetic Minority Over-Sampling Technique for addressing the common problem of class imbalance in phishing datasets. Balanced representation of phishing and benign cases in training improves the model's generalization and reduces the chance of predictions biasing.
- Balanced Performance Multi-objective optimization techniques primarily Pareto dominance and crowding distance data calculations; allow the HawkPhish-DNN cybersecurity model to optimally balance the detection rates against the false positive rates. This helps make sure they robustly perform against real world scenarios where the sensitivity and specificity trade-off are crucial for retaining user trust and minimizing disturbance.
- High level framework: The framework model proposed is modular in nature and quite extensible. The proposed DNN model framework has the potential to be extended to detect other types of cyber threats apart from phishing.

Its nature-inspired optimization design is a positive step towards intelligent and robust threat detection systems that many future researchers would want to explore.

- Superior Performance and Comprehensive Evaluation Empirical evaluations shows that HawkPhish-DNN cybersecurity model outperforms other DNN models and is not limited to academic performance. Empirical evaluations of various performance metrics accuracy, precision, recall, F1-score and AUC speak for themselves. These results reveal the effectiveness of the approach and the possibility of strengthening defenses against phishing threats.

The HawkPhish-DNN model for cyber security offers great anticipation for its application beyond the current detection and prevention of phishing attacks. As a deep learning optimization model, future work on the HawkPhish-DNN model can focus on detecting newer sophisticated phishing attacks while improving its performance, robustness, and adaptability. Such improvements can also consist of further integration of novel data sources, new neural encodings, and refinement of optimization techniques to meet higher level cyber threats. This approach shifts the focus from the shortcomings of existing detection systems to providing solutions which in turn, open the possibilities for more holistic and user-friendly defenses in cybersecurity research.

2 Related works

The ever-increasing number of phishing attacks aimed at internet users makes it imperative for researchers to prioritize phishing detection as an important field of study. There are multiple proposed solutions to this problem, such as machine learning (ML), deep learning (DL), and hybrid models.

Ozcan et al., [6] applied both character embedding and NLP features to phishing detection and achieved superior results through the use of their innovative DNN-LSTM model. In addition to outperforming traditional phishing detection techniques, this model derived the best accuracy of 99.21% compared to the finer models such as DNN (91.13%) and CNN (93.25%). Despite the promising findings, the hybrid model suffers from some limitations such as the complex combination of two different neural network types: DNN and BiLSTM. The increased computation expense makes it less efficient in real-time applications.

The new hybrid model (LR + SVC + DT) advanced by e Karim et al. [16] achieves an accuracy of 98.12% by detection of phishing URLs using soft and hard voting techniques. This model surpasses accuracy achieved from classifier trees and support vector machines individually. The downside is, with multiple classifiers and ensemble methods, there is an increased computational cost, higher training periods and less efficiency for real-time applications.

The feature based method by Das Gupta et al.[18] employs the usage of URLs and hyperlinks which led to a detection accuracy of 99.17% with XGBoost. The absence of reliance on third party systems and the capability to deal with zero-hour phishing makes this method unique. Nonetheless, this approach might be challenged with embedded objects used by phishing websites such as iFrames and images, which tend to limit the scope of more sophisticated forms of phishing. Unlike many phishing detection models, Korkmaz et al. [20] developed a system using a DNN architecture that employed a PhishTank sourced high-risk URL dataset. The system accuracy of this approach was 86.64%. The model was able to achieve these results with a comparatively basic deep learning method and lower amount of training time, but the model is still struggling with sophisticated phishing URLs which most likely need to be tackled with more intricate deep learning structures. Al-Ahmadi et al. [22] propose the PDGAN model which uses a generative adversarial network (GAN) to skirt URLs of phishing websites and determined that it was successful 97.58% of the detection accuracy. This model surpasses the detection claims of most other competitors who utilize PUCNN and PDRCNN by leveraging LSTM as a generator and CNN as a discriminator. Like all the other models, this one, however, is anchored by the usage of URL-only features, making it extremely easy to bypass for those employing advanced tactics like visual similarity and domain technique to mask their spoofed websites. According to Mourtaji et al.[25], developed a hybrid rule-based model with a convolutional neural network (CNN) capable of incorporating blacklisting, lexical methods, and behavioral analysis that earned 97.94% accuracy. This is better than what is achieved with conventional decision trees or support vector machines (SVM). The downside to this more accurate approach is the slower processing time. The combination of multiple feature extraction techniques makes real-time applications much more complex, presenting possible scalability problems. Meanwhile, Ahammad et al.[29] achieved a test accuracy score of 86.0% utilizing Light GBM, only based on the features provided to the model. Out of these features, the model incorporated domain-based and linguistic strategy which makes this method unique. The addition of another dimension of data could increase detection accuracy on malicious URLs but may impact the performance benchmark. According to Abu Khurma et al. [30] came up with a new method of feature selection FSS that utilizes Salp Swarm-SSA- based selection strategies. They were able to achieve improvements in detection accuracy up to 95.07% using reduced feature sets. This model outshines GA and PSO feature selection methods as it achieves high accuracy while reducing the features required.

Table 1 Comparative Overview of Phishing Detection Models: Techniques, Datasets, Performance and Limitations

References	Algorithm/techniques	Dataset	Performance	Limitations
[6]	Hybrid DNN–LSTM combining character embeddings and NLP features	Ebbu2017 Phish Tank	Integrates complementary feature sets for robust classification	Increased model complexity and higher computational cost
[16]	Hybrid ensemble model (combining LR, SVC, and DT with soft/hard voting) alongside various ML classifiers	PhishTank	Ensemble approach with optimized feature selection enhances performance	Increased system complexity and potential computational overhead
[18]	Hybrid feature extraction combined with machine learning (e.g., XGBoost)	PhishTank	High accuracy; eliminates third-party dependency for feature extraction	Computationally intensive hybrid feature extraction
[20]	DNN-based classification using URL-based features	PhishTank	URL-based approach reduces dependency on external content; High efficiency in accuracy and execution time	Limited quantification of results; potential dataset selection bias
[22]	GAN-based model using LSTM as generator and CNN as discriminator	MUPD PhishTank	Operates solely on URL data; high detection accuracy without external content crawling	GAN training can be complex and unstable
[25]	Hybrid rule-based approach employing CNN (with comparisons to CART, SVM, KNN)	PhishTank	Multi-perspective URL analysis enhances detection robustness	Rule-based thresholds may limit generalizability
[29]	Multiple ML classifiers (RF, DT, Light GBM, LR, SVM)	PhishTank	Comprehensive evaluation of several classifiers provides robust insights	Relies on manual feature engineering, which may limit scalability
[30]	Salp Swarm Optimization (SSA) based wrapper feature selection with k-NN classifier	PhishTank	Best performance achieved with binary SSA (X-shaped transfer functions)	Sensitive to the choice of transfer function; potential

On the other hand, this method may have scalability problems with larger sets of data because the SSA optimization process can become too costly in terms of resources and time.

Table 1 provides a concise comparative overview of various phishing detection studies, detailing the models and techniques employed, the datasets used. This summary highlights the strengths and limitations of each approach, offering valuable insights for researchers seeking advance phishing detection methodologies.

3 Methodology

The HawkPhish-DNN cybersecurity model of cybersecurity offers an innovative model for Phishing URL detection an entirely new method by combining URL advanced optimization methods with Deep Learning architectures. The methodology is comprised of four principal elements which work together to improve the model's capabilities and performance. In this section, we shall describe these components, noting how each one contributes to the proposed system's overall effectiveness. Figure 1 presents the details of the proposed model, showcasing its structure. It highlights the specifics of each stage involved. This diagram provides an in-depth view of the entire process.

3.1 Data collection and preprocessing

The selection of an appropriate data source for phishing data collection depends on the specific use case. However, the minimum required data is a list of phishing URLs. It is crucial to gather phishing webpage data promptly to ensure the information is captured in real-time or near real-time. Key data sources commonly used for this purpose include PhishTank and Openphish. Therefore, in this work two dataset are used in this paper to validate and evaluate the proposed model, the first dataset called PhishTank [31] and the other called Openphish [32]. The preprocessing stage prepares the raw URL data for analysis, extracting meaningful features and ensuring the dataset is suitable for model training.

Data collection and preprocessing are performed through a series of analytical formulations, as delineated from Eq. 1 to Eq. 18. These equations systematically normalize and transform the raw input features, ensuring they are optimally configured for the next stage. This rigorous preprocessing is fundamental in ensuring both the accuracy and robustness of the phishing detection methodology for the proposed model.

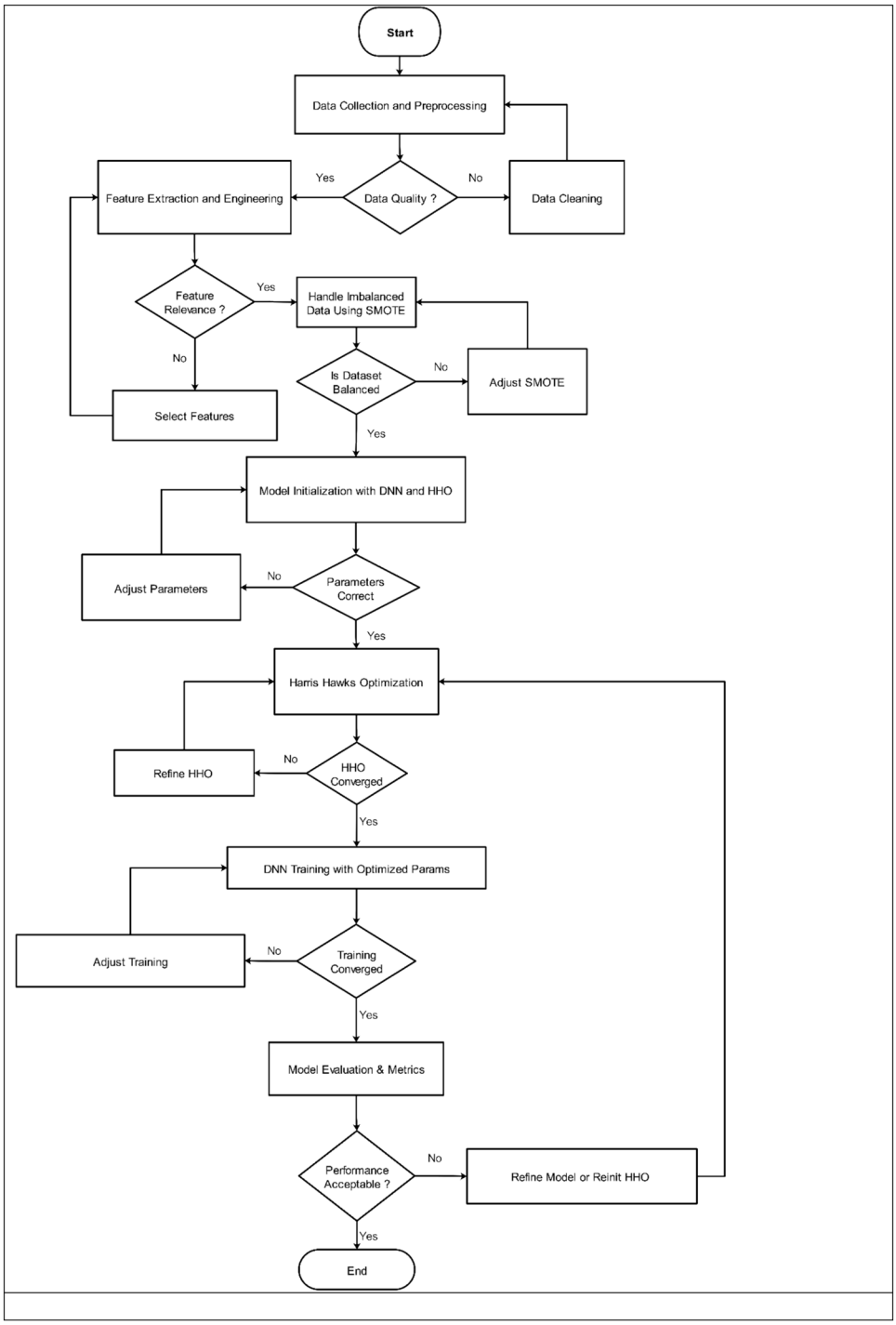


Fig. 1 Overview of the Proposed HawkPhish-DNN Cybersecurity Model: Steps and Stages

3.1.1 Data representation

Combine the PhishTank and Openphish datasets into a unified dataset D. The Eq. (1) represents the unification of the PhishTank and Openphish datasets into a single dataset D, where each URL d_i is indexed from 1 to n. Combining the datasets ensures that all URLs are processed uniformly in subsequent steps, which is essential for building a robust model capable of distinguishing between phishing and benign URLs. This step can be represented by the following equation:

$$\left.\begin{aligned} D_{PhishTank} &= \{d_i^{PhishTank}\}_{i=1}^{n_{PhishTank}} \\ D_{Openphish} &= \{d_j^{Openphish}\}_{j=1}^{n_{Openphish}} \\ D_{benign} &= \{d_j^{benign}\}_{j=1}^{n_{benign}} \end{aligned}\right\rangle D \\ = D_{PhishTank} \cup D_{Openphish} \cup D_{benign} = \{d_i\}_{i=1}^{n} \tag{1}$$

where the $n = n_{PhishTank} + n_{Openphish} + n_{benign}$, refer to the number of URLs entries in each dataset, while the variables d_{1i} and d_{2j} represent the URL entry in each dataset.

3.1.1.1 Data cleaning The function $unique(\cdot)$ filters out duplicate entries from the dataset D, resulting in a cleaned dataset D' containing only unique URLs which represented by Eq. (2). This step is crucial to prevent biases and overfitting that could result from multiple instances of the same URL influencing the model disproportionately.

$$D' = unique(D) \tag{2}$$

3.1.1.2 URL normalization The function $normalize(\cdot)$ standardizes each URL d_i by applying transformations such as converting to lowercase, removing unnecessary parameters, decoding encoded characters, and standardizing protocols. Normalization ensures consistency across all URLs, which is essential for accurate tokenization and feature extraction in subsequent steps. Normalize each URL to a standard format:

$$d_i' = normalize(d_i), \forall d_i \in D' \tag{3}$$

3.1.2 Feature extraction

Extract numerical features that capture essential characteristics of each URL.

3.1.2.1 Domain feature extraction

- Number of Subdomains: This feature counts the number of subdomains present in the URL by utilizing the Eq. (4). Phishing URLs often include multiple subdomains to mimic legitimate sites or to obscure the true domain, making this a valuable indicator.

$$subdomain_count_i = count_subdomains(domain_i) \tag{4}$$

where $domain_i$ is the domain part of the normalized URL d_i' .and $count_subdomains(.)$ is a function that counts the number of subdomains in the domain.

- Domain Length: The length of the domain can be indicative of phishing attempts by utilizing the Eq. (5), as malicious URLs may use excessively long domain names to confuse users or hide malicious intent.

$$domain_len_i = length(domain_i) \tag{5}$$

where $length(.)$ is a function that calculates the number of characters in the domain.

3.1.2.2 URL length calculation The length of each normalized URL crucial for identifying phishing URLs, is computed using the below equation:

$$len_i = length(d_i') \tag{6}$$

This feature measures the total number of characters in the normalized URL d_i'. Phishing URLs may be unusually long to include deceptive information or to hide malicious scripts within the URL.

3.1.2.3 Entropy calculation Compute the entropy of each URL to measure character randomness.

- Character Probability:

$$p_c = \frac{f_c}{L_i} \tag{7}$$

where f_c is the frequency of character c in d_i' and L_i is the length of d_i'. The probability p_c represents how often each character c appears in the URL. This probability is used to calculate the entropy, reflecting the randomness or complexity of the URL's character distribution.

- Entropy:

$$H_i = -\sum_{c \in chars(d_i')} p_c log_2(p_c) \tag{8}$$

where $chars(d_i')$ is the set of unique characters in the URL d_i'.

Entropy H_i quantifies the randomness or unpredictability of the URL's character sequence. High entropy may indicate obfuscation techniques often used in phishing URLs to avoid detection by simple pattern-matching algorithms.

3.1.3 Feature vector construction

The feature vector x_i combines the numerical features extracted from the URL d_i'. These features provide valuable information for the model to learn patterns associated with phishing URLs, such as unusual domain structures or high character randomness. Construct the feature vector x_i for each URL:

$$x_i = [subdomain_count_i, domain_len_i, len_i, H_i] \tag{9}$$

3.1.4 Label encoding

Each URL d_i is assigned to a label y_i, where 1 represents a phishing URL and 0 represents a benign URL. This binary labeling is essential for supervised learning, allowing the model to distinguish between the two classes during training. Assign numerical labels to the URLs for supervised learning:

$$y_i = \begin{cases} 1 \text{ if } d_i \text{ is a phishing URL} \\ 0 \text{ if } d_i \text{ is a benign URL} \end{cases} \tag{10}$$

3.1.5 Handling imbalanced data

SMOTE addresses class imbalance by generating synthetic samples of the minority class (usually phishing URLs), ensuring that the model does not become biased toward the majority class and can learn equally from both classes. Therefore, SMOTE is used to balance the dataset:

$$F, Y = SMOTE(\{x_i\}, \{y_i\}) \tag{11}$$

where F is indicated to the augmented feature set with balanced classes and Y is indicated to the corresponding labels after balancing.

3.1.6 Tokenization and text analysis

This section describes URL tokenization, TF-IDF vectorization and vector representation.

3.1.6.1 URL tokenization The *tokenize*(.) function splits the normalized URL d_i' into smaller components or tokens based on delimiters such as slashes, dots, and hyphens. Tokenization facilitates textual analysis by isolating meaningful elements like domain names, file paths, and parameters. The bellow equation show the Tokenize each normalized URL:

$$tokens_i = tokenize(d_i') \tag{12}$$

3.1.6.2 TF-IDF vectorization Compute the TF-IDF representation for each token in the URLs.

- Term Frequency (TF): TF measures how frequently a token t_j appears in a URL relative to the total number of tokens in that URL.

$$TF(t_j, d_i) = \frac{count(t_j, d_i)}{\sum_{t' \in d_i} count(t', d_i)} \tag{13}$$

where t_j A specific token. $count(t_j, d_i)$ is a number of times token t_j appears in URL d_i. The denominator sums the counts of all tokens in d_i .

- Inverse Document Frequency (IDF): IDF evaluates the importance of a token by measuring how common or rare it is across all URLs. Rare tokens receive higher IDF values, highlighting their significance in distinguishing phishing URLs.

$$IDF(t_j) = \log\left(\frac{N}{df(t_j)}\right) \tag{14}$$

where $df(t_j)$ number of URLs containing the tokent_j. N Total number of URLs in the dataset.

- TF-IDF Calculation: TF-IDF combines TF and IDF to assign a weight to each token, reflecting its importance within a specific URL and across the entire dataset. Tokens that are frequent in a URL but rare across all URLs get higher weights, aiding in identifying unique patterns.

$$TF - IDF(t_j, d_i) = TF(t_j, d_i) \times IDF(t_j) \tag{15}$$

3.1.6.3 Vector representation The TF-IDF vector v_i captures the textual content of the URL in numerical form, enabling the model to analyze and learn from textual patterns that may indicate phishing. by creating the TF-IDF vector for each URL:

$$v_i = [TF - IDF(t_1, d_i), TF - IDF(t_2, d_i), ...TF - IDF(t_m, d_i)] \tag{16}$$

Here, v_i is the resulting vector for each URL, m is the number of unique tokens in the dataset, and each vector element represents the TF-IDF value for a specific token.

3.1.7 Scaling and normalization

Scaling and normalization adjust the features to have a mean of zero and a standard deviation of one. This ensures that all features contribute equally to the model training process and prevents features with larger scales from dominating. The standardize the feature vectors by the following equation:

$$v'_{ij} = \frac{v_{ij} - \mu_j}{\sigma_j} \tag{17}$$

where v_{ij} The $j-th$ feature of the $i-th$ URL. μ_j is the mean of the $j-th$ feature across all URLs. σ is the standard deviation of the $j-th$ feature.

3.1.8 Final dataset

The final dataset F contains all the preprocessed and normalized features, providing a robust input for the DNN. Finally, the final preprocessed dataset is compiled:

$$F = \{v_i'\}_{i=1}^{n'} \tag{18}$$

where n' is the total number of processed URLs after applying SMOTE. F is a set of standardized feature vectors ready for model training.

3.2 Feature engineering and optimization:(harris hawks optimization (HHO))

The Harris Hawks Optimization (HHO) algorithm represents one of the latest nature-inspired optimization techniques, modeled after the cooperative hunting strategies of Harris hawks. Within the HawkPhish-DNN cybersecurity model, HHO plays a pivotal role in refining the parameters of the Deep Neural Network namely, its weights and biases [33]. By emulating the dynamic and intelligent behaviors of hawks during the hunt, this algorithm efficiently explores the search space, mitigating the risks of getting trapped in local minima and accelerating the convergence process. Furthermore, by balancing exploration and exploitation, HHO enhances the model's accuracy in detecting phishing URLs. Its application therefore serves as a critical step toward improving overall model performance and reliability in defending against phishing attacks.

3.2.1 Population initialization

Each hawk represents a potential solution (a set of DNN parameters) based on Eq. (19). Population diversity allows the algorithm to explore various regions of parameter space effectively to initialize a population of hawks for representing candidate DNN parameter sets.

$$X_i = \left[w_{i1}, w_{i2}, ..., w_{iD}\right], i = 1, ..., N \tag{19}$$

where X_i position (parameter set) of the $i-th$ hawk. w_{ij} the $j-th$ weight or bias parameter of the DNN in the $i-th$ hawk. D Total number of DNN parameters (weights and biases). And N Total number of hawks in the population.

3.2.1.1 Dynamic penalty function A dynamic penalty function is employed to manage constraint violations in the optimization problem which can be represented by Eq. (20), guiding the search toward feasible solutions. The penalty function, $P(X, t)$, is defined to adapt dynamically based on the solution's adherence to constraints.

$$P(X, t) = \sum_{i=1}^{m} \max(0, g_i(X_i)).(\frac{t}{T})^{\alpha(t)} \tag{20}$$

where:

- X_i: The solution vector (position of a hawk in the context of the HHO algorithm), representing the candidate solution at the current iteration.
- $g_i(X_i)$: The $i-th$ constraint function applied to the solution vector X_i.
- α: A user-defined parameter that controls the growth rate of the penalty. This parameter influences how the penalty changes as optimization progresses. The Eq. (21) shows how to calculate the value of α:

$$\alpha(t) = \alpha_{start} + (\alpha_{end} - \alpha_{start})(\frac{t}{T}) \tag{21}$$

3.2.1.2 Penalty coefficient The dynamic parameter that adjusts the severity of penalties for constraint violations during optimization can be represent by Eq. (22), evolving with each iteration. This gradual increase in stringency ensures the algorithm increasingly focuses on feasible solutions, enhancing the quality of outcomes.

$$\lambda(t) = \lambda_0(\frac{t}{T})^{\beta} \tag{22}$$

where:

- λ_0: The initial value of the penalty coefficient. This is a user-defined parameter that sets the base level of penalty applied at the beginning of the optimization process.
- β: A user-defined parameter that controls the growth rate of the penalty coefficient which can be calculated by Eq. (23). Therefore, this parameter determines how quickly the penalty coefficient increases as the optimization process progresses.

$$\beta = \begin{cases} if\, \beta > 1,\ \beta = High \\ if\, \beta = 1,\ \beta = Moderate \\ if\, \beta < 1,\ \beta = Low \end{cases} \tag{23}$$

Where:

- The High value of β indicate for strict constraint enforcement towards the end.
- The Moderate value of β indicate for a balanced approach.
- The Low value of β indicate for early constraint satisfaction with flexibility towards the end.

The $\lambda(t)$ increases progressively with each iteration, encouraging the optimization to prioritize feasible solutions by applying stricter penalties for constraint violations. This adaptive approach allows flexibility in early stages while ensuring stricter adherence to constraints later, thereby enhancing efficiency.

3.2.2 Exploration and exploitation phase

This section discusses both the exploration and exploitation phase.

3.2.2.1 Exploration phase To update hawk positions based on stochastic behaviors Eq. (24) is used. This equation defines the hawks' movement during exploration, allowing them to explore the search space broadly by moving away from random hawks or towards the average position:

$$X_i(t+1) = \begin{cases} X_{rand}(t) - r_1|X_{rand}(t) - 2r_2X_i(t)| - \lambda(t).P(X,t)\ q \geq 0.5 \\ (X_{best}(t) - X_m(t)) - r_3(LB + r_4(UB - LB)) - \lambda(t).P(X,t),\ q < 0.5 \end{cases} \tag{24}$$

where:

t Current iteration.
$X_{rand}(t)$: Random hawk's position at iteration t.
$X_{best}(t)$: Best hawk's position at iteration t.
$X_m(t)$: Mean position of the hawks at iteration t which can be computed by using the equation number ().
$r_1, r_2, r_3, r_4 \sim \cup(0,1)$: Random coefficients uniformly distributed between 0 and 1.
$q \sim \cup(0,1)$: Random variable determining the strategy.
LB, UB: Lower and upper bounds of the search space.
$\lambda(t)$: The penalty coefficient (from the previous phase) at iteration t. It adjusts the severity of penalties applied to constraint violations.
$P(X,t)$: The penalty function that quantifies the degree of constraint violation for the position X_i at iteration t.

3.2.2.2 Computing the mean position The mean position $X_m(t)$ represents the center of the population's current positions which is calculated by Eq. (25), providing a reference point for exploration strategies.

$$X_m(t) = \frac{1}{N}\sum_{i=1}^{N} X_i(t) \tag{25}$$

where:

- N is the number of hawks (solutions) in the population.
- $X_i(t)$ is the position of the $i-th$ hawk at iteration t.

3.2.2.3 Escaping energy and jump strength Calculate the escaping energy E and jump strength J:

- Escaping Energy E:

The escaping energy E simulates the prey's ability to escape which can be represented by Eq. (26), influencing whether the hawks should focus on exploration or exploitation.

$$E = 2E_0(1 - \frac{t}{T}),\ E_0 \sim \mathrm{U}(-1,1) \tag{26}$$

where E_0 is a random number in the range $[-1,1]$, t is the current iteration, and T is the maximum number of iterations.

- Jump Strength J:

The jump strength J can be represented by Eq. (27) which determines the step size during the hawk's movement toward the prey (optimal solution), introducing variability in their approach.

$$J = 2(1 - r),\ r \sim U(0,1) \tag{27}$$

where r is a random number in the range [0,1].

3.2.2.4 Exploitation phase Update positions based on the exploitation strategy. Meanwhile, the position update mechanism depends on the escaping energy E. The Eq. (28) is used to calculate the general form of $\Delta X(t)$ during the exploitation phase.

$$\Delta X(t) = X_{best}(t) - X_i(t) \tag{28}$$

Depending on the value of E, the hawk may engage in soft besiege, hard besiege, or a combination with progressive rapid dives.

In this phase, the Harris Hawks Optimization (HHO) algorithm shifts focus from exploration to exploitation, aiming to refine solutions and converge on the best possible outcome. The exploitation phase is divided into two sub-phases: Soft Besiege and Hard Besiege, depending on the energy level E.

- Soft Besiege ($if|E| \geq 0.5$):

The **soft besiege** occurs when the hawks are trying to surround the prey (best solution) gradually, without committing a direct attack. The update rule for the hawk's position in this case is calculated based on Eq. (29).

$$X_i(t+1) = \Delta X(t) - E|JX_{best}(t) - X_i(t)| - \lambda(t).P(X,t) \quad (29)$$

where:

$\Delta X(t)$: The difference between the current and previous position of the hawk.
E: The escaping energy, controlling the strength of the hawk's movement.
J: A random factor affecting the hawk's movement toward the best position.
X_{best}: The best-known solution at iteration t.
$P(X,t)$: The dynamic penalty function applied for constraint violations.
$\lambda(t)$: The penalty coefficient that adjusts the severity of the penalties.

When the prey (optimal solution) has sufficient energy to escape ($if|E| \geq 0.5$,), hawks perform a soft besiege, cautiously approaching the prey to prevent it from escaping. Therefore, in the soft besiege, the hawk adjusts its position based on its distance from the best solution, aiming to slowly surround it. The randomness introduced by J ensures some exploration while gradually homing in on the target. The penalty term ensures constraint handling throughout the process.

- *Hard Besiege* ($if|E| < 0.5$)

The hard besiege represents a more aggressive attack on the best solution. The hawk commits to a more direct approach to catch the prey, updating its position according to Eq. (30).

$$X_i(t+1) = X_{best}(t) - E|\Delta X(t)| - \lambda(t).P(X,t) \quad (30)$$

where:

- X_{best}: The best solution at the current iteration.
- E: Escaping energy, controlling the hawk's speed and intensity.
- $\Delta X(t)$: The difference in position from previous iterations.
- $P(X,t)$: The dynamic penalty function.
- $\lambda(t)$: The penalty coefficient.

When the prey is exhausted ($if|E| < 0.5$), hawks perform a hard besiege, aggressively moving toward the prey to capture it quickly. In the hard besiege, the hawk moves directly toward the best solution with more intensity, using less randomness. This phase focuses on exploiting the best solution more aggressively, converging quickly on it while still considering constraint violations through penalties.

After the besiege phase, the algorithm evaluates whether the solution can be improved further or if convergence has been reached. Depending on the result, the algorithm may return to exploration or continue exploiting the current best solutions. The exploitation phase in HHO is unique in its ability to adapt between soft and hard besiege modes based on the escaping energy E. This flexibility allows the hawks to balance between cautious refinement (soft besiege) and aggressive convergence (hard besiege). This dual-mode strategy enhances the algorithm's ability to avoid local optima and efficiently converge on the global optimum, while also incorporating constraint handling through penalties.

3.2.2.5 Progressive rapid dives Levy Flight introduces sudden, large steps in the hawk's movement, allowing the algorithm to escape local optima and explore new areas. To enhance exploitation using Levy Flight (LF):

- LF(D)

$$LF(D) = 0.01 \times \frac{u\sigma}{|v|^{1/\beta}}, u, v \sim N(0,1),\ \beta = 1.5 \quad (31)$$

where u, v random variables from a normal distribution. σ scaling parameter. β levy distribution parameter.

- Update Positions:

The hawk considers both Y and Z as potential next positions, choosing the one that offers better fitness through Eqs. (32,33).

$$Y = X_{best}(t) - E|JX_{best}(t) - X_i(t)| - \lambda(t).P(X,t) \quad (32)$$

$$Z = Y + S.LF(D) - \lambda(t).P(X,t) \quad (33)$$

where Y position after exploitation moves. Z position after applying for Levy Flight. S scaling factor for the Levy Flight step which computed adaptively by the equation:

$$S(t) = \sigma.(1 - \frac{t}{T}).U(0,1) \quad (34)$$

The reason for making S adaptive is that it can be modified to decrease over time, ensuring larger steps in the early stages of optimization (encouraging exploration) and smaller steps in the later stages (encouraging exploitation).

- Selecting Next Position

The hawk updates its position based on Eq. (35) to Y or Z if either improves upon its current position. If neither Y nor Z is better, the hawk remains at $X_i(t)$.

$$X_i(t+1) = \begin{cases} Y, \text{ if } f(Y) < f(X_i(t)) \\ Z, \text{ if } f(Z) < f(X_i(t)) \\ X_i(t), \text{ otherwise} \end{cases} \quad (35)$$

where $f(.)$ fitness function evaluating the quality of the solution.

3.3 Integration of HHO with DNN

Integrating the HHO algorithm with the Deep Neural Network forms the core innovation of the HawkPhish-DNN cybersecurity model. This integration involves formulating the network's parameter optimization as a multi-objective problem, aiming to maximize detection accuracy while minimizing the false positive rate. The HHO algorithm is employed to search for optimal parameter sets that satisfy these objectives. By treating the weights and biases of the DNN as variables within the HHO framework, the model leverages the algorithm's global search capabilities to find superior solutions that traditional gradient-based methods might miss. This integration enhances the network's ability to generalize from the training data and improves its resilience against sophisticated phishing strategies.

The HHO algorithm optimizes the DNN parameters to maximize detection accuracy and minimize false positives, effectively tuning the model for better performance.

3.3.1 Objective functions

- Maximize Detection Accuracy (f_1): The function $f_1(X_i)$ calculates the detection accuracy of the DNN when using the parameter set X_i which can be calculated by Eq. (36). Higher accuracy indicates better performance in correctly identifying phishing URLs.

$$f_1(X_i) = \frac{Numberofcorrectlyclassifiedphishing URLs}{Total\ number\ of\ phishing\ URLs} \quad (36)$$

- Minimize False Positive Rate (f_2): The function $f_2(X_i)$ computes the false positive rate of the DNN with parameters X_i which can be calculated by Eq. (37). A lower false positive rate means fewer benign URLs are incorrectly classified as phishing, which is critical for user trust and system efficiency.

$$f_2(X_i) = \frac{Numberofincorrectlyclassifiedbenign URLs}{Total\ number\ of\ benign\ URLs} \quad (37)$$

3.3.2 Multi-objective optimization

Formulate the optimization problem can be represented by Eq. (38)

$$\text{Minimize f}(X_i) = [-f_1(X_i), f_2(X_i)] \quad (38)$$

The negative sign in $-f_1(X_i)$ converts the maximization problem into a minimization one, aligning both objectives. The goal is to find a parameter set X_i that simultaneously minimizes the negative accuracy (maximizes accuracy) and the false positive rate.

- *Pareto Dominance*: A solution X_i is considered better than X_j if it is not worse in all objectives and strictly better in at least one which can be represented by Eq. (39). Pareto dominance helps identify optimal trade-offs between conflicting objectives.

$$\text{X}_\text{i}\ dominates\ X_j\ if\ \forall k, f_k(X_i) \leq f_k(X_j)\ and\ \exists k, f_k(X_i) < f_k(X_j) \quad (39)$$

- *Crowding Distance*: The crowding distance $d(X_i)$ is represented by Eq. (40) which measures the density of solutions around X_i in the objective space. It ensures diversity among the solutions by favoring those in less crowded areas.

$$d(X_i) = \sum_{k=1}^{m} \left(\frac{f_k(X_{i+1}) - f_k(X_{i-1})}{f_k^{max} - f_k^{min}} \right) \quad (40)$$

3.4 DNN Construction, Training, and Hyperparameter Tuning (Deep Neural Network Structure)

Phishing URLs and normal URLs are classified using a Deep Neural Network (DNN) which serves as the main classification module integrated with a feature extraction unit in a preprocessing step. During the training phase, the network is given a standardized feature vector, and it tries to update its parameters to reduce the loss function. The DNN architecture includes activation and several hidden layers to learn data patterns that are complex and non-linear. When the model is also embedded with HHO-based parameters, it achieves high accuracy and sufficient balance. The model is evaluated through detection accuracy and false positive rates which are important in assessing the effectiveness of the model in real life scenarios. To summarize, HHO optimized parameters are employed in DNN for proficient URL classification ensuring effectiveness in phishing detection.

3.4.1 Network structure

- Input Layer: The input layer is represented by Eq. (41) which receives standardized feature vectors representing each URL.

$$x_i = [x_{i1}, x_{i2}, ..., x_{in}] \tag{41}$$

where x_i input feature vector for URL i and n number of features extracted during preprocessing.

- Hidden Layer(s): The hidden layers transform the input data through learned weights and biases which can be represented by Eq. (42), allowing the network to capture complex patterns in the data. For each layer l:

$$z_j^{(l)} = ReLU\left(\sum_i w_{ij}^{(l)} z_i^{(l-1)} + b_j^{(l)}\right) \tag{42}$$

where:

$z_j^{(l)}$ Activation of neuron j in layer l.
$w_{ij}^{(l)}$ Weight connecting neuron i in layer $l-1$ to neuron j in layer l.
$b_j^{(l)}$ Bias term for neuron j in layer l.
$ReLU(\cdot)$ Activation function introducing non-linearity.
Output Layer: The output layer produces the final prediction for each URL which can be represented by Eq. (43), indicating the likelihood of it being a phishing URL.

$$\hat{y}_i = \sigma\left(\sum_j w_{kj}^{(L)} z_j^{(L-1)} + b_k^{(L)}\right) \tag{43}$$

Where:

$\hat{y}_i$ predicted probability that URL i is phishing.
L The last layer of the network.
σ Sigmoid activation function mapping outputs to probabilities between 0 and 1.

3.4.2 Activation functions

- Sigmoid Activation: it compresses input values into a range between 0 and 1, suitable for binary classification tasks, and it can be calculated by Eq. (44).

$$\sigma(x) = \frac{1}{1 + e^{-x}} \tag{44}$$

- ReLU Activation: it introduces non-linearity to the network and helps mitigate the vanishing gradient problem by allowing gradients to flow when the input is positive, and it can be calculated by Eq. (45).

$$ReLU(x) = max(0, x) \tag{45}$$

3.4.3 Loss Function

The loss function quantifies the error between the predicted probabilities and the actual labels which can be represented by Eq. (46), guiding the optimization of the network parameters. Use binary cross-entropy loss to measure the discrepancy between predicted and actual labels:

$$L = -\frac{1}{N}\sum_{i=1}^{N} [y_i log(\hat{y}_i) + (1 - y_i) log(1 - \hat{y}_i)] \tag{46}$$

where:

N Total number of URLs.
y_i is the true label of URL i.
$\hat{y}_i$ is the predicted probability for URL i.

3.4.4 Weight and Bias Updates

To update the DNN parameters using gradient descent, the Eqs. (47, 48) are used.

- Weight Update:

$$w_{ij} \leftarrow w_{ij} - \eta \frac{\partial L}{\partial w_{ij}} \tag{47}$$

- Bias Update:

$$b_j \leftarrow b_j - \eta \frac{\partial L}{\partial b_j} \tag{48}$$

where:

η Learning rate controlling the step size during optimization.
$\frac{\partial L}{\partial w_{ij}}, \frac{\partial L}{\partial b_j}$ Gradients of the loss function with respect to weights and biases.

These equations adjust the network's parameters in the direction that minimizes the loss function, improving the model's performance over time.

3.5 Enhanced algorithm integration

The final stage in the methodology is to fully integrate the HHO-optimized DNN with other advanced techniques to boost performance even further. This is done by employing advanced feature extraction techniques such as entropy

measurement and TF-IDF vectorization to increase the input data's representational ability. To mitigate the effects of imbalanced datasets, SMOTE is applied so that the model learns from both phishing and benign instances equally. Also, the multi-objective optimization approach is enhanced by adding concepts of Pareto dominance and crowding distance calculations to facilitate solution diversity and avoid premature convergence. The DNN is optimally tuned to work with a phishing detection system by applying the most effective solutions from the HHO algorithm which yields an extremely accurate and reliable system with low false positive rates. This proposed HawkPhish-DNN cybersecurity model surpasses conventional models and significantly impacts the cybersecurity field.

3.5.1 Selection of best solutions

- Best Phishing Detection Solution $X_{phish}(t)$: Eq. (49) is used to select the parameter set X_i that yields the highest detection accuracy, focusing on correctly identifying phishing URLs.

$$X_{phish}(t) = \arg\max_{X_i} f_1(X_i) \tag{49}$$

- Best Benign URL Detection Solution $X_{benign}(t)$: Eq. (50) is used to select the parameter set X_i that results in the lowest false positive rate, emphasizing the correct classification of benign URLs.

$$X_{benign}(t) = \arg\min_{X_i} f_2(X_i) \tag{50}$$

- The Overall Best Solution $X_{best}(t)$: The overall best solution X_{best} is a weighted combination of the best phishing detection and benign detection solutions, balancing both objectives to achieve optimal performance. The Eq. (51) is used for calculating dynamic weighted combination strategy.

$$X_{best}(t) = w_{phish}X_{phish}(t) + w_{benign}X_{benign}(t) \tag{51}$$

where w_{phish} and w_{benign} are weighting factors such that $w_{phish} + w_{benign} = 1$.

The performance of the proposed model is maximize based on the current detection accuracy and false-positive rate by updated the weight dynamically via using Eqs. (52, 53).

$$w_{phish}(t) = \frac{f_1(X_{best}(t))}{f_1(X_{best}(t)) + f_2(X_{best}(t))} \tag{52}$$

$$w_{benign}(t) = \frac{f_2(X_{best}(t))}{f_1(X_{best}(t)) + f_2(X_{best}(t))} \tag{53}$$

4 Experiments and results

To evaluate the effectiveness of the HawkPhish-DNN cybersecurity model in detecting phishing URLs, a series of experiments were conducted using a comprehensive dataset comprising both phishing and benign URLs. The experiments aimed to assess the model's performance in terms of detection accuracy, false positive rate, and other relevant metrics. This section details the experimental setup, the results obtained, and an analysis of the model's efficiency.

4.1 Experimental setup

This section presents dataset preparation, model configuration and evaluation metrics.

4.1.1 Dataset preparation

- *Data Sources*:
  - *Phishing URLs*: Collected from PhishTank [31] and OpenPhish [32] repositories, ensuring a diverse range of phishing strategies and obfuscation techniques.
  - *Benign URLs*: Obtained from Alexa's top-ranked websites and publicly available datasets of legitimate URLs.

4.1.2 Model configuration

- *Deep Neural Network (DNN)*:
  - *Architecture*: Three hidden layers with 128, 64, and 32 neurons, respectively.
  - *Activation Functions*: ReLU for hidden layers, Sigmoid for the output layer.
  - *Loss Function*: Binary cross-entropy.
  - *Optimization Algorithm*: Harris Hawks Optimization (HHO) algorithm integrated for parameter optimization.
- *Harris Hawks Optimization (HHO)*:
  - *Population Size*: 30 hawks.
  - *Maximum Iterations*: 100
  - *Objective Functions*: Maximizing detection accuracy and minimizing false positive rate.

4.1.3 Evaluation metrics

To comprehensively assess the model's performance, the following metrics were employed:

- *Accuracy*: Proportion of correctly classified URLs.
- *Precision*: Proportion of correctly predicted phishing URLs among all predicted phishing URLs.
- *Recall*: Proportion of correctly predicted phishing URLs among all actual phishing URLs.
- *F1-Score*: Harmonic mean of precision and recall.
- *False Positive Rate*: Proportion of benign URLs incorrectly classified as phishing.
- *Area Under the ROC Curve*: Measures the model's ability to distinguish between classes.
- *Confusion Matrix*: Provides a detailed breakdown of true positives, true negatives, false positives, and false negatives.

As illustrated in Fig. 2 and Table 2, the HawkPhish-DNN cybersecurity model demonstrates superior performance metrics, highlighting its effectiveness in accurately classifying phishing and benign URLs.

Figure 2 and Table 2 indicate that the HawkPhish-DNN cybersecurity model demonstrates exceptional performance with a True Positive (TP) rate of 99.40%, ensuring that nearly all phishing URLs are accurately detected. It maintains a remarkably low False Positive (FP) rate of 0.20% and a False Negative (FN) rate of 0.60%, minimizing incorrect classifications. Additionally, the True Negative (TN) rate of 99.80% highlights the model's high reliability in correctly identifying benign URLs, underscoring its superiority in delivering precise and trustworthy phishing detection.

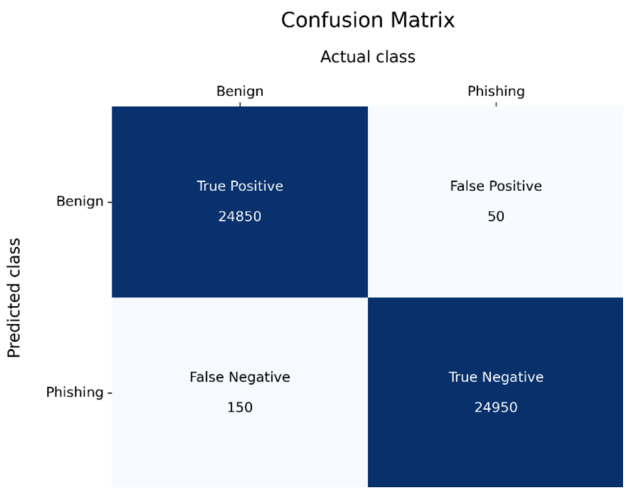


Fig. 2 Confusion Matrix of the Proposed HawkPhish-DNN Cybersecurity Model

Table 2 the contents of confusion matrix for the proposed HawkPhish-DNN cybersecurity model

Model	TP (%)	FP (%)	FN (%)	TN (%)
HawkPhish-DNN	99.40	0.20	0.60	99.80

4.2 Comparative analysis of model performance

The efficacy of the proposed HawkPhish-DNN cybersecurity model is rigorously evaluated against a traditional Deep Neural Network (DNN) without optimization. The comparative results, as illustrated in Table 3, demonstrate a substantial enhancement across all key performance metrics, underscoring the superiority of the HawkPhish-DNN cybersecurity model in phishing URL detection.

Table 3 provides an analysis of the outcome results achieved by the classical Deep Neural Network (DNN) model in comparison with the HawkPhish-DNN cybersecurity model regarding several performance indices. Its accuracy stands at 97.9%, which increases to 99.6% with the HawkPhish-DNN cybersecurity model a 1.7% improvement. This improvement indicates a more dependable classification capability, which means the chances of phishing URLs being misclassified to benign URLs and the other way around is decreased. The precision of the model, ranging from 98.1% in the traditional DNN to 99.8% in the HawkPhish-DNN cybersecurity model, reflects the changes in the accuracy of positive identifications made by the model. This improvement means that there is an increased reduction of false positives where authentic URLs are wrongly marked as phishing URLs. Further, recall, callable as the true positive rate of the model, increases from 97.7% to 99.4%, improving the capture rate of harmful URLs while enabling the model to resist phishing attacks more effectively. Also, the F1 score emerged from 97.9% to the proposed model value of 99.6%. This increase demonstrates that the boost in performance was neither biased towards recall nor precision. Finally, the Area Under the Curve (AUC), a composite indicator of the

Table 3 indicates the values for Accuracy, Precision, Recall, F1-score, and AUC for the proposed HawkPhish-DNN cybersecurity model based on the PhishTank and OpenPhish datasets

Model	Accuracy	Precision	Recall	F1score	AUC
Traditional DNN (without HHO)	97.9	98.1	97.7	97.9	0.988
HawkPhish-DNN (proposed)	99.6	99.8	99.4	99.6	0.998

model's capability of differentiation increased from 0.988 to 0.998.

This improvement highlights the immense strength of the HawkPhish-DNN cybersecurity model at differentiating phishing and non-phishing URLs at multiple threshold levels. In general, the HawkPhish-DNN cybersecurity model surpassed the conventional DNN for each measure assessed, realizing an increase in accuracy, precision, recall, and F1-score, together with an AUC of 0.998. These collective achievements endorse the model's increased strength and dependability in the area of phishing detection; thus, this greatly enhances cybersecurity because it makes available an efficient and precise tool to defend against phishing attacks.

4.3 Discussion

This section discusses potential factors with the proposed model like local optima, feature engineering, and comparison with traditional models.

4.3.1 Impact of HHO integration

The integration of the HHO algorithm significantly contributes to the model's superior performance:

- *Escaping Local Optima*: The HHO algorithm's exploration capabilities prevent the model from getting trapped in local minima.
- *Adaptive Parameter Tuning*: Dynamic adjustment of optimization parameters leads to better convergence and generalization.

4.3.2 Importance of feature engineering

- *Entropy Measures*: Capturing the randomness in URLs helps detect obfuscated phishing attempts.
- *TF-IDF Vectorization*: Semantic analysis of URL components enhances the model's ability to identify phishing patterns.

4.3.3 Comparison with traditional models

- *Traditional DNN vs. HawkPhish-DNN:* The HawkPhish-DNN cybersecurity model's optimization leads to a notable improvement over a traditional DNN without HHO.

The results validate that the HawkPhish-DNN cybersecurity model is effective in phishing URL detection. It boasts an impressive URL detection accuracy of 99.6% alongside a false positive rate of just 0.2%, outperforming traditional techniques. The use of advanced feature extraction techniques and the innovative integration of the Harris Hawks Optimization algorithm contribute significantly to these results. The model's ability to maintain high precision and recall ensures reliable detection, essential for practical cybersecurity applications.

4.4 Comparative analysis of hawkphish-dnn cybersecurity model against existing models

Table 4 comparison of the proposed HawkPhish-DNN cybersecurity model with the recent models in the arts under the same condition such as same dataset " PhishTank and Openphish" as well as the same evaluation metrics to be the comparison fair.

Table 4 presents a comprehensive comparison of the proposed HawkPhish-DNN cybersecurity model against several state-of-the-art models documented in the existing literature. This comparison is meticulously conducted under uniform conditions, utilizing the same datasets PhishTank and OpenPhish and consistent evaluation metrics, namely Accuracy, Precision, Recall, F1-score, and Area Under the Curve (AUC). Such an approach ensures the fairness and reliability of comparative analysis. The HawkPhish-DNN cybersecurity model distinctly outperforms all benchmarked models across nearly all evaluated metrics. Specifically, it achieves an Accuracy of 99.6%, surpassing the highest reported accuracy of 99.21% by the DNN-BiLSTM model [6]. In terms of Precision, HawkPhish-DNN cybersecurity model records an outstanding 99.8%, marginally exceeding

Table 4 Performance Comparison of HawkPhish-DNN cybersecurity model with Existing Phishing Detection Models on PhishTank and OpenPhish Datasets

Ref	Model	Accuracy	Precision	Recall	F1score	AUC
[20]	DNN approach	86.64	86.06	87.24	86.64	None
[29]	LightGBM	89.5	None	None	None	None
[30]	BSSA-X	95.07	None	None	None	None
[22]	PDGAN	97.58	98.02	97.27	97.64	None
[25]	Hybrid Rule-Based	97.94	None	None	None	None
[16]	Hybrid ML	98.12	97.31	96.33	95.89	None
[18]	Hybrid anti-phishing	99.17	99.51	98.81	99.15	99.89
[6]	DNN-BiLSTM	99.21	None	None	99.41	99.34
Proposed HawkPhish-DNN		99.6	99.8	99.4	99.6	99.8

the 99.51% precision of the Hybrid anti-phishing model [18]. The Recall metric for HawkPhish-DNN cybersecurity model stands at 99.4%, which is notably higher than the 98.81% achieved by the same Hybrid model. Furthermore, the F1-score of 99.6% for HawkPhish-DNN cybersecurity model eclipses the 99.41% reported by the DNN-BiLSTM model [6]. While the Hybrid anti-phishing model [18] exhibits a slightly superior AUC of 99.89 compared to HawkPhish-DNN's 99.8, the proposed model compensates with superior performance across all other critical metrics. This near-perfect AUC underscores HawkPhish-DNN's exceptional capability to distinguish between phishing and benign URLs, albeit with a marginally lower AUC compared to the Hybrid anti-phishing model. Moreover, several models in the literature, such as LightGBM [30] and BSSA-X [30], report incomplete metric sets, limiting their comparability. Nevertheless, among the models with comprehensive metric reporting, HawkPhish-DNN cybersecurity model consistently demonstrates superior or comparable performance, affirming its robustness and efficacy in phishing detection.

In summary, the HawkPhish-DNN cybersecurity model not only achieves the highest Accuracy, Precision, Recall, and F1-score but also maintains a competitive AUC, thereby establishing itself as the most effective solution among the compared models. These results highlight the model's advanced optimization and feature extraction capabilities, positioning it as a leading approach in the ongoing battle against phishing threats in the cybersecurity landscape.

5 Conclusion

The HawkPhish-DNN cybersecurity model makes a significant step forward in the detection of phishing URLs by implementing the Harris Hawks Optimization algorithm into a Deep Neural Network. The novel architecture allows for efficient optimization of the networks parameters which improves the model's response to learning complex structures associated with phishing attacks. The experimental results show that the model performs superbly, as it was able to reach a detection accuracy of 99.6% while achieving an extremely low Rate of False Positives, equal to 0.2%. These results confirm both claims how the model is proficient in classifying phishing URLs and how the websites are not phishing attempts are barely misclassified. The HawkPhish-DNN cybersecurity model also demonstrates tremendous effectiveness when countering modern challenges in adequate proactive prevention of cyber threats due to the fact that it employs enhanced feature extraction methods for modeling sophisticated phishing attempts and balances several optimization goals. Future Work will include widening the scope of the detected cyber threats for the HawkPhish-DNN cybersecurity model to include malware distribution and spear-phishing attacks. Also, the implementation of real-time detection and the application of the model in large-scale real-life settings will be investigated. Exploring the fusion of other nature-inspired optimization techniques and neural network architectures could also improve the model's adaptability and performance to counter the ever-evolving range of possible cyber-attacks.

Funding Open Access funding enabled and organized by CAUL and its Member Institutions. Victoria University (AU), Ayman Ibaida, Read and Publish Agreement.

Data availability Not applicable.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

References

1. Asiri S, Xiao Y, Alzahrani S, Li S, Li T (2023) A survey of intelligent detection designs of HTML URL phishing attacks. IEEE Access 11:6421–6443
2. Al Ogaili, R.R.N., Alomari, E.S., Alkorani, M.B.M. et al. Malware cyberattacks detection using a novel feature selection method based on a modified whale optimization algorithm. Wireless Network, Volume 30, Pages 7257–7273, (2024). https://doi.org/10.1007/s11276-023-03606-z
3. Abu-Amara F, Al Hosani R, Al Tamimi H, Al Hamdi B (2024) Spreading cybersecurity awareness via gamification: zero-day game. Int J In Technol 16(5):2945–2953
4. Prabakaran MK, MeenakshiSundaram P, Chandrasekar AD (2023) An enhanced deep learning-based phishing detection mechanism to effectively identify malicious URLs using variational autoencoders. IET Inf Secur 17(3):423–440
5. S. Bell and P. Komisarczuk, "An analysis of phishing blacklists: Google safe browsing, openphish, and phishtank," in Proceedings of the Australasian Computer Science Week Multiconference, 2020, pp. 1–11.
6. Ozcan, A., Catal, C., Donmez, E. et al. A hybrid DNN–LSTM model for detecting phishing URLs. Neural Computing and Applications, Volume 35, Pages 4957–4973, (2023). https://doi.org/10.1007/s00521-021-06401-z
7. Nuiaa RR, Alsaeedi AH, Manickam S, Al-Shammary DEJ, Alsaeedi AH, Al-Shammary DEJ (2022) Evolving dynamic fuzzy clustering (EDFC) to enhance DRDoS_DNS attacks detection mechnism. Int J Intell Eng Syst 15(1):509–5019. https://doi.org/10.22266/ijies2022.0228.46
8. Raja AS, Vinodini R, Kavitha A (2021) Lexical features based malicious URL detection using machine learning techniques. Mater Today Proc 47:163–166

9. El Aassal A, Baki S, Das A, Verma RM (2020) An in-depth benchmarking and evaluation of phishing detection research for security needs. Ieee Access 8:22170–22192
10. Rajak A, Tripathi R (2024) DL-SkLSTM approach for cyber security threats detection in 5G enabled IIoT. Int J Inf Technol 16(1):13–20
11. Sakib Shahriar Shafin, An explainable feature selection framework for web phishing detection with machine learning, Data Science and Management, Volume 8, Issue 2, 2025, Pages 127-136, ISSN 2666-7649. https://doi.org/10.1016/j.dsm.2024.08.004
12. Aighuraibawi AHB et al (2024) Hybridizing flower pollination algorithm with particle swarm optimization for enhancing the performance of IPv6 intrusion detection system. Alexandria Eng J 104:504–514
13. Arash Mahboubi, Khanh Luong, Hamed Aboutorab, Hang Thanh Bui, Geoff Jarrad, Mohammed Bahutair, Seyit Camtepe, Ganna Pogrebna, Ejaz Ahmed, Bazara Barry, Hannah Gately, Evolving techniques in cyber threat hunting: A systematic review, Journal of Network and Computer Applications, Volume 232, 2024, 104004, ISSN 1084-8045. https://doi.org/10.1016/j.jnca.2024.104004
14. Usoh M, Asuquo P, Ozuomba S, Stephen B, Inyang U (2023) A hybrid machine learning model for detecting cybersecurity threats in IoT applications. Int J Inf Technol 15(6):3359–3370
15. A. Basit, M. Zafar, A. R. Javed, and Z. Jalil, "A novel ensemble machine learning method to detect phishing attack," in 2020 IEEE 23rd International Multitopic Conference (INMIC), IEEE, 2020, pp. 1–5.
16. Karim A, Shahroz M, Mustofa K, Belhaouari SB, Joga SRK (2023) Phishing detection system through hybrid machine learning based on URL. IEEE Access 11:36805–36822
17. Hadi SM, Alsaeedi AH, Dohan MI, Nuiaa RR, Manickam S, Alfoudi ASD (2022) Dynamic evolving cauchy possibilistic clustering based on the self-similarity principle (DECS) for enhancing intrusion detection system. Int J Intell Eng Syst 15(5):252–260. https://doi.org/10.22266/ijies2022.1031.23
18. Das Guptta S, Shahriar KT, Alqahtani H, Alsalman D, Sarker IH (2024) Modeling hybrid feature-based phishing websites detection using machine learning techniques. Ann Data Sci 11(1):217–242
19. Abdul Samad SR et al (2023) Analysis of the performance impact of fine-tuned machine learning model for phishing URL detection. Electronics 12(7):1642
20. M. Korkmaz, E. Kocyigit, O. K. Sahingoz, and B. Diri, "Deep neural network based phishing classification on a high-risk url dataset," in International conference on soft computing and pattern recognition, Springer, 2020, pp. 648–657.
21. Mughaid A, ALZu'bi S, Hnaif A, Taamneh S, Alnajjar A, Elsoud EA (2022) An intelligent cyber security phishing detection system using deep learning techniques. Cluster Comput 25(6):3819–3828
22. Al-Ahmadi S, Alotaibi A, Alsaleh O (2022) PDGAN: Phishing detection with generative adversarial networks. Ieee Access 10:42459–42468
23. Sabahno M, Safara F (2022) ISHO: improved spotted hyena optimization algorithm for phishing website detection. Multimed Tools Appl 81(24):34677–34696
24. Do NQ, Selamat A, Krejcar O, Herrera-Viedma E, Fujita H (2022) Deep learning for phishing detection: taxonomy, current challenges and future directions. Ieee Access 10:36429–36463
25. Mourtaji Y, Bouhorma M, Alghazzawi D, Aldabbagh G, Alghamdi A (2021) Hybrid rule-based solution for phishing URL detection using convolutional neural network. Wirel Commun Mob Comput 2021(1):8241104
26. Sunkara S, Suresh T, Sathiyasuntharam V (2023) Red fox optimizer based feature selection with optimal deep learning based Intrusion detection for network security. Int J Inf Technol 15(8):4437–4447
27. I. Skula and M. Kvet, 2024 "A Framework for Preparing a Balanced and Comprehensive Phishing Dataset." IEEE Access.
28. Abi Sen AA (2022) A comprehensive privacy and security framework for dynamic protection (CPSF). Int J Inf Technol 14(5):2477–2485
29. Ahammad SKH et al (2022) Phishing URL detection using machine learning methods. Adv Eng Softw 173:103288
30. R. A. Khurma, K. E. Sabri, P. A. Castillo, and I. Aljarah, (2021) "Salp swarm optimization search based feature selection for enhanced phishing websites detection," in Applications of Evolutionary Computation: 24th International Conference, EvoApplications 2021, Held as Part of EvoStar 2021, Virtual Event, April 7–9, 2021, Proceedings 24, Springer pp. 146–161.
31. PhishTank, "What is Phishing?," PhishTank. [Online]. Available: https://phishtank.org/what_is_phishing.php. Accessed 19 Aug 2024.
32. OpenPhish, "OpenPhish," OpenPhish.. [Online]. https://openphish.com
33. Heidari AA, Mirjalili S, Faris H, Aljarah I, Mafarja M, Chen H (2019) Harris hawks optimization: algorithm and applications. Futur Gener Comput Syst 97:849–872

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.